

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Artificial Intelligence and Application Development

Practical – II

Roll No.: T032	Name: KHUSHI
Class: TYIT	Batch: 2
Date of Assignment: 9-07-26	Date/Time of Submission: 9-07-26

Q1] Given an initial , Configuration of the 8-Puzzle and a Goal Configuration , write a python program to find the shortest sequence of moves to reach the goal state using Breadth – First Search (BFS).

CODE:-

```
# library
from collections import deque

# Function to display puzzle state
def print_state(state):
    for i in range(0, 9, 3):
        print(state[i], state[i + 1], state[i + 2])
    print()

# Function to find possible next states
def get_neighbors(state):
    neighbors = []

# Find position of blank tile
    blank_index = state.index(0)

# Possible moves: Up, Down, Left, Right
    moves = {
        "Up": -3,
        "Down": 3,
        "Left": -1,
        "Right": 1
    }

    for move, position_change in moves.items():
        new_index = blank_index + position_change

# Check valid moves
        if move == "Up" and blank_index < 3:
            continue
        if move == "Down" and blank_index > 5:
            continue
        if move == "Left" and blank_index % 3 == 0:
            continue
        if move == "Right" and blank_index % 3 == 2:
            continue

# Create new state by swapping blank tile
        new_state = list(state)
        new_state[blank_index], new_state[new_index] = (
            new_state[new_index],
            new_state[blank_index] )
        neighbors.append((tuple(new_state), move))
    return neighbors

# BFS function
def bfs(initial_state, goal_state):
    queue = deque()
    # Each queue item contains: current_state, path
    queue.append((initial_state, []))
    visited = set()
    visited.add(initial_state)
    while queue:
        current_state, path = queue.popleft()
```

```

# Check if goal is reached
if current_state == goal_state:
    return path

# Generate next possible states
for neighbor, move in get_neighbors(current_state):
    if neighbor not in visited:
        visited.add(neighbor)
        queue.append((neighbor, path + [(move, neighbor)]))
return None

# Main Program
initial_state = (
    1, 2, 3,
    4, 0, 6,
    7, 5, 8
)
goal_state = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)
print("Initial State:")
print_state(initial_state) print("Goal State:")
print_state(goal_state)
solution = bfs(initial_state, goal_state)
if solution:
    print("Shortest sequence of moves:")
    current_step = 1
    for move, state in solution:
        print("Step", current_step, "- Move:", move)
        print_state(state)
        current_step += 1
    print("Goal reached successfully.")
    print("Total number of moves:", len(solution))
else:
    print("No solution found.")
print("Khushii T032")

```

OUTPUT:-

```

Initial State:
1 2 3
4 0 6
7 5 8

Goal State:
1 2 3
4 5 6
7 8 0

```

```

Shortest sequence of moves:
Step 1 - Move: Down
1 2 3
4 5 6
7 0 8

Step 2 - Move: Right
1 2 3
4 5 6
7 8 0

Goal reached successfully.
Total number of moves: 2
Khushii T032

```

Q2] Given two water jugs of 4 Litres and 3 Litres capacity, write a python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS) .[VARY CAPACITY OF JUGS] .

CODE:-

Water Jug Problem using DFS

```
def dfs(cap1, cap2, target):
    stack = [(0, 0)] # Initial state
    visited = set()
    while stack:
        state = stack.pop()
        if state in visited:
            continue
        visited.add(state)
        print(state)
        x, y = state
```

Check if target is reached

```
if x == target or y == target:
    print("\nTarget achieved:", state)
    return
```

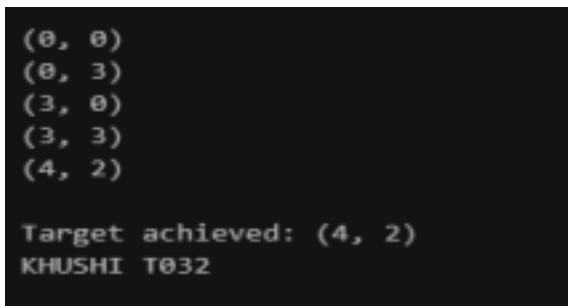
Possible moves

```
next_states =
[
    (cap1, y), # Fill Jug 1
    (x, cap2), # Fill Jug 2
    (0, y), # Empty Jug 1
    (x, 0), # Empty Jug 2
    (max(0, x-(cap2-y)), min(cap2, x+y)), # Pour Jug1 -> Jug2
    (min(cap1, x+y), max(0, y-(cap1-x))) # Pour Jug2 -> Jug1
]
for s in next_states:
    if s not in visited:
        stack.append(s)
print("No solution found.")
```

Input from user

```
cap1 = int(input("Enter capacity of Jug 1: "))
cap2 = int(input("Enter capacity of Jug 2: "))
target = int(input("Enter target amount: "))
dfs(cap1, cap2, target)
print("KHUSHI T032")
```

OUTPUT:-



```
(0, 0)
(0, 3)
(3, 0)
(3, 3)
(4, 2)

Target achieved: (4, 2)
KHUSHI T032
```

Q3] Given a weighted graph representing cities and distance between them , write a python program to find the least – Cost Path from Arad to Bucharest using Uniform Cost Search (UCS)

CODE:-

```
import heapq
graph =
```

```
{
    'Mumbai': [('Pune', 150), ('Hyderabad', 710)],
    'Pune': [('Mumbai', 150), ('Bangalore', 840)],
    'Hyderabad': [('Mumbai', 710), ('Bangalore', 570)],
    'Bangalore': []
}
```

```
def ucs(start, goal):
    queue = [(0, start, [start])]
    visited = []

    while queue:
        cost, city, path = heapq.heappop(queue)

        if city == goal:
            return cost, path

        if city not in visited:
            visited.append(city)

            for neighbor, distance in graph[city]:
                heapq.heappush(
                    queue,
                    (
                        cost + distance,
                        neighbor,
                        path + [neighbor]
                    )
                )

    return None, None
```

```
cost, path = ucs("Mumbai", "Bangalore")
```

```
print("Least-cost path:")
print("Path:", " -> ".join(path))
print("Total Distance:", cost, "km")
print("KHUSHI T032")
```

OUTPUT:-

```
Least-cost path:
Path: Mumbai -> Pune -> Bangalore
Total Distance: 990 km
KHUSHI T032
```